

Anno Accademico 2025/2026

Leonardo Donati

Ingegneria del Software  
**Reverse Proxy HTTP**

*Tesina.*



**UNIMORE**  
UNIVERSITÀ DEGLI STUDI DI  
MODENA E REGGIO EMILIA

# Sommario

Questo elaborato descrive la progettazione di un reverse proxy HTTP realizzato in Java, in grado di leggere le regole di instradamento da un *file di configurazione esterno* e di inoltrare ogni richiesta al servizio corretto sulla base del percorso richiesto. Oltre alla funzionalità di base di instradamento, il sistema è stato progettato per supportare più istanze dello stesso servizio a una singola rotta, introducendo la necessità di un meccanismo di *bilanciamento del carico*.

# Indice

|          |                                                   |          |
|----------|---------------------------------------------------|----------|
| <b>1</b> | <b>Introduzione</b>                               | <b>3</b> |
| <b>2</b> | <b>Analisi dei Requisiti</b>                      | <b>3</b> |
| 2.1      | Requisiti Funzionali . . . . .                    | 3        |
| 2.2      | Requisiti Non Funzionali . . . . .                | 3        |
| <b>3</b> | <b>Stile Architetturale</b>                       | <b>4</b> |
| 3.1      | Perché non uno Stile Event-Driven . . . . .       | 4        |
| 3.2      | I Livelli del Sistema . . . . .                   | 4        |
| 3.3      | Linguaggio . . . . .                              | 4        |
| 3.4      | Configurazione Dichiarativa delle Route . . . . . | 4        |
| <b>4</b> | <b>Modellazione UML</b>                           | <b>5</b> |
| 4.1      | Usa-Case Diagram . . . . .                        | 5        |

# 1 Introduzione

Nei moderni sistemi web raramente un singolo servizio è responsabile della gestione di tutte le richieste in arrivo: applicazioni frontend, API REST, servizi di autenticazione e risorse statiche sono spesso ospitati su processi distinti. Un reverse proxy si colloca all'ingresso di questa infrastruttura, intercettando le richieste dei client e instradandole verso il servizio backend corretto, nascondendo al client la reale topologia del sistema.

## 2 Analisi dei Requisiti

### 2.1 Requisiti Funzionali

- **Routing delle Richieste HTTP:** il sistema deve ricevere una richiesta HTTP dal client e instradarla verso il servizio backend corretto, sulla base del percorso richiesto e delle regole definite in un file di configurazione esterno.
- **Load Balancing:** quando una rotta è associata a più istanze dello stesso servizio, il sistema deve distribuire le richieste tra le istanze disponibili secondo una strategia configurabile (ad esempio round-robin, selezione casuale, ecc), senza dover modificare il codice sorgente per cambiarla.
- **Risoluzione tramite Referer:** per le richieste di risorse statiche (stili css, script, immagini) prive di una ruota esplicita, deve essere necessario risalire al servizio di destinazione analizzando l'header **Referer** della richiesta, così da individuare la pagina che le ha originate.
- **Configurazione Dichiarativa delle Route:** deve essere possibile definire, tramite un file YAML esterno, l'insieme delle route disponibili e i servizi backend ad esse associati, senza dover ricompilare l'applicazione.
- **Gestione di Connessioni Concorrenti:** il sistema deve poter gestire più richieste client in maniera concorrente, ciascuna elaborata su un thread dedicato prelevato da un pool.

### 2.2 Requisiti Non Funzionali

- **Basso Overhead:** il tempo impiegato dal proxy per instradare una richiesta deve essere trascurabile rispetto al tempo di risposta del servizio backend.
- **Estendibilità delle Strategie di Bilanciamento:** deve essere possibile introdurre nuovi algoritmi di balancing del carico con il minor impatto possibile sulle classi esistenti.
- **Scalabilità della Configurazione:** l'aggiunta di nuovi servizi, istanze o route deve richiedere la sola modifica del file di configurazione.
- **Robustezza:** un errore nella comunicazione con un singolo backend o l'assenza di una route corrispondente, non deve compromettere la disponibilità del proxy per le altre richieste in corso.
- **Portabilità:** il software deve poter essere eseguito su qualsiasi piattaforma con una Java Virtual Machine, senza dipendenze dal sistema operativo sottostante.

### 3 Stile Architetture

Il reverse proxy è distribuito come un singolo processo Java, eseguito come un'unica unità deployabile. Non è un sistema distribuito, perché la logica applicativa, dall'accettazione delle connessioni fino all'inoltro della risposta, risiede all'interno della JVM. Per questo motivo, lo stile architetture adottato è quello del **Monolite a Livelli**, in cui le responsabilità del sistema sono suddivise in livelli logici ben distinti, ciascuno dei quali dipende solo dai livelli sottostanti attraverso interfacce astratte.

#### 3.1 Perché non uno Stile Event-Driven

I reverse proxy utilizzati a livello enterprise sono tipicamente realizzati con un'architettura **Event-Driven**, basata su eventi e I/O non bloccante, capace di gestire un numero molto elevato di connessioni concorrenti. Ho comunque scelto di adottare il Monolite a Livelli per questo progetto, perché questa struttura è più facile da progettare, testare e descrivere rispetto a una asincrona a eventi. Inoltre, con il numero di richieste concorrenti di questo prototipo, un thread per connessione è la soluzione migliore, perché i benefici di scalabilità di uno stile event-driven diventano determinanti solo con carichi molto più elevati. L'utilizzo di un modello event-driven, con I/O non bloccante, può comunque essere un possibile sviluppo futuro del progetto.

#### 3.2 I Livelli del Sistema

- **Livello di Connessione:** accetta le connessioni TCP in ingresso tramite il **ServerSocket** principale e delega la gestione di ciascuna connessione a un thread worker prelevato da un pool.
- **Livello di Parsing:** responsabile dell'analisi dello stream raw di byte proveniente dal socket del client, traducendo l'header HTTP in un oggetto che contiene metodo, URL e altri header in maniera strutturata.
- **Livello di Logica di Routing e Balancing:** incapsula la logica applicativa principale. In base alla route del percorso, seleziona il servizio nel backend corretto, usando la strategia di balancing dichiarata.
- **Livello di Configurazione:** gestisce le regole di instradamento caricate dal file YAML esterno, fornendo un'interfaccia agli altri livelli del sistema.

#### 3.3 Linguaggio

Il linguaggio utilizzato è Java. La scelta è motivata dalla disponibilità di API per la gestione di socket e connessioni TCP, da un modello di concorrenza basato su thread semplice da utilizzare tramite un **ExecutorService** e dall'esistenza di librerie per il parsing dei file di configurazione, come **SnakeYAML** per i file YAML.

Trattandosi di un programma che introduce un ulteriore passaggio tra client e servizio, l'overhead introdotto dalla JVM rispetto a linguaggi più performanti come C/C++ risulta pienamente accettabile.

#### 3.4 Configurazione Dichiarativa delle Route

La scelta di inserire le regole di instradamento in un file YAML esterno, caricato all'avvio dell'applicazione, evita che l'aggiunta o la rimozione di un servizio richieda una modifica del codice sorgente.

##### YAML Esempio di File di Configurazione

```
strategy: round-robin
```

```
routes:
  - path: /
    service:
      - host: localhost
        port: 8081
      - host: localhost
        port: 8082
  - path: /api
    service:
      - host: localhost
        port: 9090
```

Per ogni route, viene specificato il percorso, un host e una porta. Possono esserci più servizi associati a una route, per questo, il campo **strategy**, permette di scegliere l'algoritmo di load balancing da applicare senza intervenire sul codice.

## 4 Modellazione UML

Principali diagrammi che descrivono struttura e il comportamento dinamico del software.

### 4.1 Usa-Case Diagram

- **Client / Browser:** l'entità che invia una richiesta HTTP al proxy.
- **Amministratore:** definisce le rotte nel file di configurazione.
- **Servizio Backend:** il sistema esterno che riceve la richiesta inoltrata ed elabora la risposta.